

You can travel through the nodes of your tree anyway you want but we will look at preorder, inorder, postorder traversals.

→ in these traversals we can do any operations at every node, we will simply look at print value of node when visiting every node.

Preorder traversal :-

Visit current node first then visit left subtree in preorder fashion visit right subtree in preorder fashion.

(or) simply

Preorder(root) → print(root.value); preorder(root.left);  
preorder(root.right);

inorder(root) → inorder(left subtree); print(root.value);  
inorder(right subtree);

Postorder(root) → postorder(left subtree); postorder(right subtree);  
print(root.value)

Preorder → 

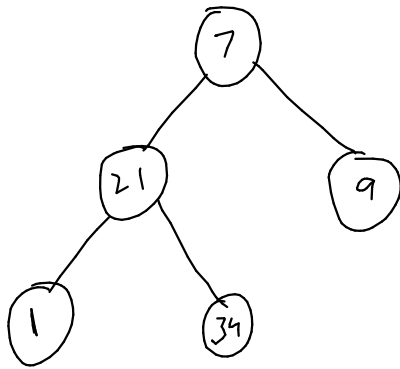
inorder → 

Postorder → 

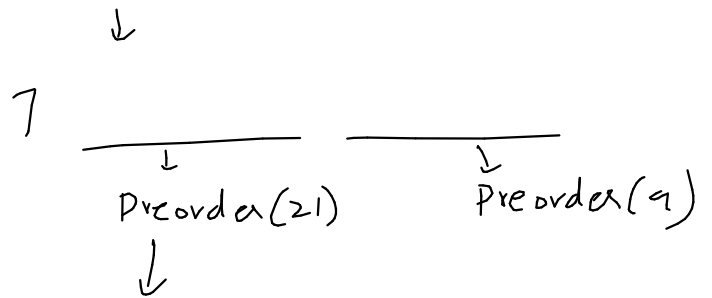
Here we are recursively calling for both left, right subtrees.

So stopping condition is, if root is null, don't do anything.

ex:- write the preorder traversal for following tree.



Preorder(7)



→ if your root is a single node with no children,

7, 21, 1, 34, 9

Preorder, inorder, postorder of root will be root itself. → because our stopping condition is if root == null, return;

Similarly inorder =

```

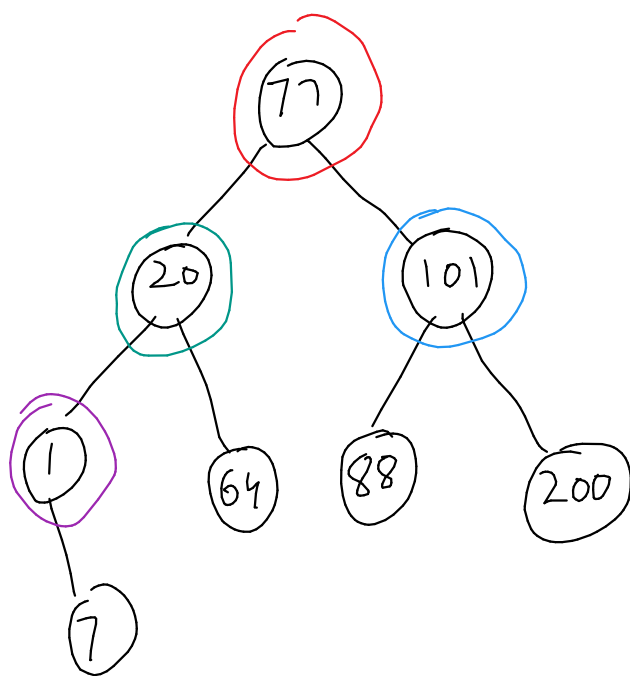
      7
    ____
   21  7  9
  ____
 1, 21, 34, 7, 9
  
```

Postorder =

```

      7
    ____
   21  9  7
  ____
 1, 34, 21, 9, 7
  
```

ex-2:-



Pre order    77, 20, 1, 7, 64, 101, 88, 200

inorder    1, 7, 20, 64, 77, 88, 101, 200

Post order    7, 1, 64, 20, 88, 200, 101, 77

→ Try to understand based on colors, 77 circled with red coz its root and 77's left and right subtrees underlined using red. Same for all other colors.

→ look at more examples for clarity.

Given the (preorder, inorder) or (postorder, inorder)  
we can construct our tree back from  
these 2 orders.

→ We cannot do this with given (pre, postorders)  
only

Why not? i explained in class. Think  
for yourself after reading  
next part.

From preorder, inorder

→ First number in preorder will always be the  
root of the tree (or) subtree.

→ Identify where that node is lying in the inorder  
and left of that will be inorder of left subtree and  
right will be right subtree's inorder.

→ recursively construct from preorder, inorders  
of each subtree.

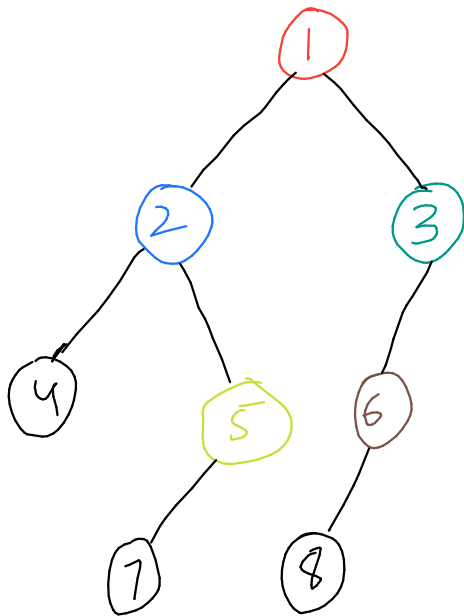
example :- Construct tree from below preorders, inorders.

Pre order

①, ②, 4, ⑤, 7, ③, ⑥, 8

inorder

4, ②, 7, ⑤, ①, 8, ⑥, ③



→ all root nodes we identified are marked with different colors.

→ Single nodes are written in black.

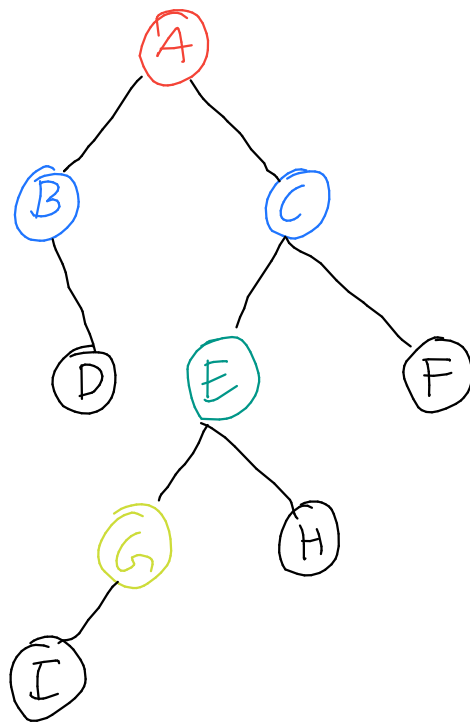
Example - 2 :-

Preorder :

① A ② B D ③ C ④ E ⑤ G I H F

inorder :

① B D ② A ③ I ④ G ⑤ E H ⑥ C F



## From Post order, Inorder

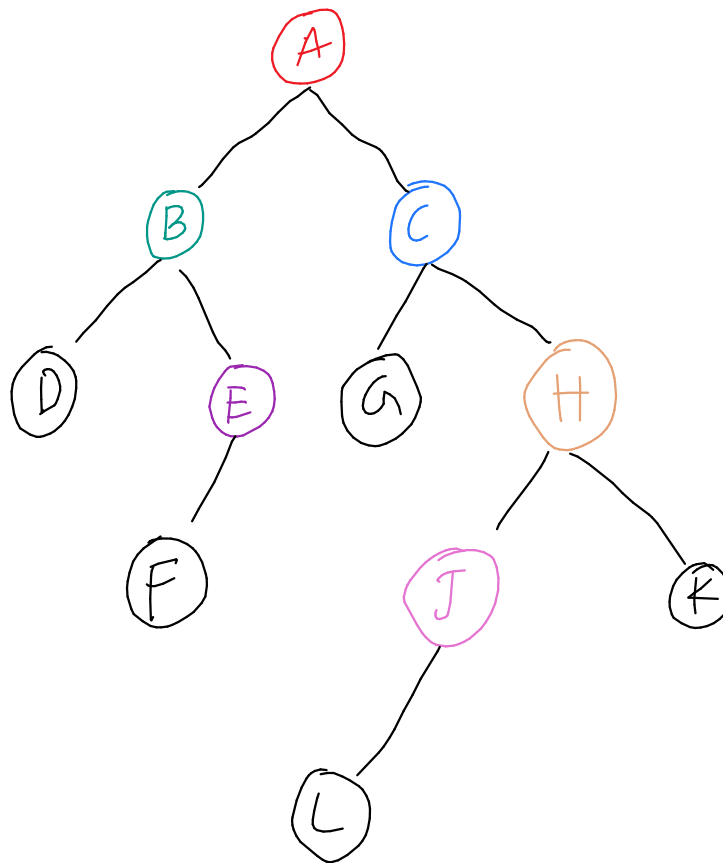
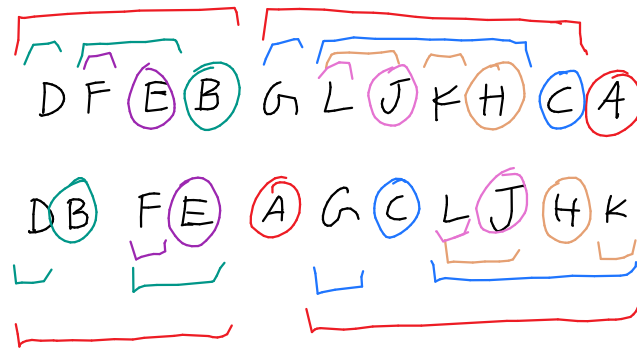
→ Similar to preorder, In postorder root nodes appear at right end. Find where that lies inside inorder.

And recursively Identify left, right subtrees and continue.

example :-

Post order

inorder



→ Again, Identified root nodes are marked in color.

→ Single leaf nodes are just written in black.

→ Please meet me if you need more clarity.

About implementing pre order, post, in orders,

look at the java code i gave in the  
github site.

Run the programs yourself.

Try constructing trees of your own,  
implement them in main function instead  
of the tree i wrote.

And run to verify your pre, in, post orders  
of your trees.